API Security with Tyk: Authentication Methods

Learn the different ways to verify and secure access to your APIs in Tyk

Tyk

AuthN and AuthZ

- Tyk Gateway acts as a secure intermediary between clients and services
- Each API proxy can define authentication requirements
- Controls request routing and access enforcement

Authentication vs. Authorization

- Authentication (AuthN): Confirms identity (who)
- Authorization (AuthZ): Grants permissions (what)
- Work together to secure access

- Prevent unauthorized access
- Protect data integrity
- Meet compliance requirements

How AuthN and AuthZ Works in Tyk

- Middleware chain processes each request
- Multiple authentication methods supported
- Custom authentication plugins supported
- Auth methods can be chained, though order is not configurable

Auth Tokens

- A bearer token is an access token that allows any party in possession of it (a “bearer”) to access associated resources.
- No cryptographic proof of ownership is required — possession is sufficient for access.
- Because of this, bearer tokens must be handled securely:
    - In transit: Use HTTPS to protect against interception.
    - At rest: Avoid logging or persisting in storage unless properly encrypted.
- Typically transmitted in the Authorization **header**, possibly prefixed with Bearer, but can also be sent via **query parameters** or **cookies**.
- **Optional Enhancements:**
    - **Dynamic mTLS**: Clients can provide TLS certificates instead of tokens.
    - **Signature Validation**: Add signature-based validation for backwards compatibility (e.g., MasherySHA256/MD5).

```
Authorization: 58dbe0dbfe2f5a0b7af7f7d08cd4e31304414e994ff724126
```

Tyk

Auth Tokens

Auth Token Location and Advanced Options in Tyk

OpenAPI's `securitySchemes` defines a single token location:

```
components:
  securitySchemes:
    myAuthScheme:
      type: apiKey
      in: header
      name: Authorization
```

Tyk expands this by allowing additional token locations:

```
x-tyk-api-gateway:
  server:
    authentication:
      enabled: true
      securitySchemes:
        myAuthScheme:
          enabled: true
          query:
            enabled: true
            name: query-auth
          cookie:
            enabled: true
            name: cookie-auth
```

Basic Authentication

Basic Authentication is a simple method where credentials (username and password) are sent to the server, typically via an HTTP header.

Credentials are combined as username:password, then base64 encoded:

```
Basic base64Encode(username:password)
```

```
Authorization: Basic am9obkBzbWl0aC5jb206MTIzNDU2Nw==
```

Security Warning:

- Credentials are sent as base64-encoded plain text
- Susceptible to interception
- **Use only with TLS/mTLS for additional protection**

Tyk

Basic Authentication

Extracting Credentials from the Request Payload

- Some APIs, like SOAP, may include credentials in the request body instead of the HTTP headers.
- Tyk can handle this using regular expressions to extract credentials directly from the body.

```
x-tyk-api-gateway:
  server:
    authentication:
      enabled: true
      securitySchemes:
        myAuthScheme:
          enabled: true
        extractCredentialsFromBody:
          enabled: true
          userRegexp: '<User>(.*)</User>'
          passwordRegexp: '<Password>(.*)</Password>'
```

Regex patterns must capture only one group — the actual value of the username and password.

Basic Authentication

Tyk does not generate Basic Auth credentials.

- You must manually create and register usernames and passwords.

This is done by creating a Tyk key:

- Includes a username and password
- Grants access to the protected API

Important:

The API key itself is not used directly by the client.

Clients must send Basic Auth credentials (username:password) in the request.

HMAC

- Stands for Hash-Based Message Authentication Code
- Adds security by requiring a signature with each request
- Uses a secret key that is never sent over the network

Client creates a signature using a date header and a secret key:

Base64Encode(HMAC-SHA1("date: Mon, 02 Jan 2006 15:04:05 MST", secret_key))

request is sent with an Authorization header:

Authorization: Signature keyId="hmac-key-1",algorithm="hmac-sha1",signature="Base64(HMAC-SHA1(signing string))“

HMAC

- Tyk verifies the request:
  - Extracts the keyId from the header
  - Retrieves the secret key from Redis
  - Recreates the HMAC signature based on the request's date header
  - If the generated signature matches the one in the request, access is granted
- Creating HMAC keys is the same as creating regular access tokens
- Tyk generates a secret key for the key owner to be stored

HMAC

Upstream Signing

- Tyk takes the request it’s about to send upstream.
- It creates an HMAC signature of key parts of that request:
  - (request-target) → the method and path.
  - All the headers in the request.
- If there’s no Date header, Tyk adds one automatically (because it’s required by the HMAC signing standard).

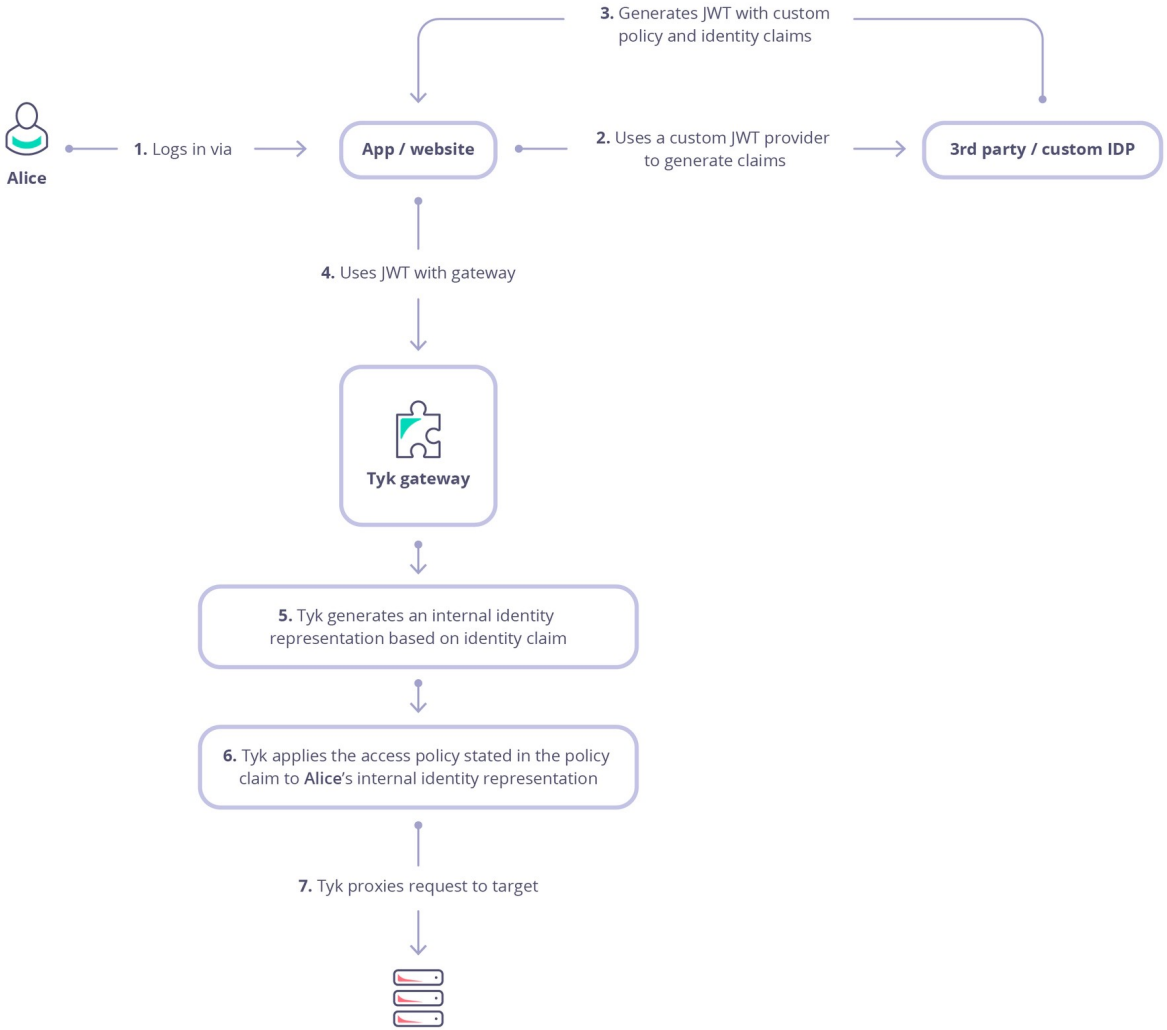
JSON Web Tokens

JSON web tokens

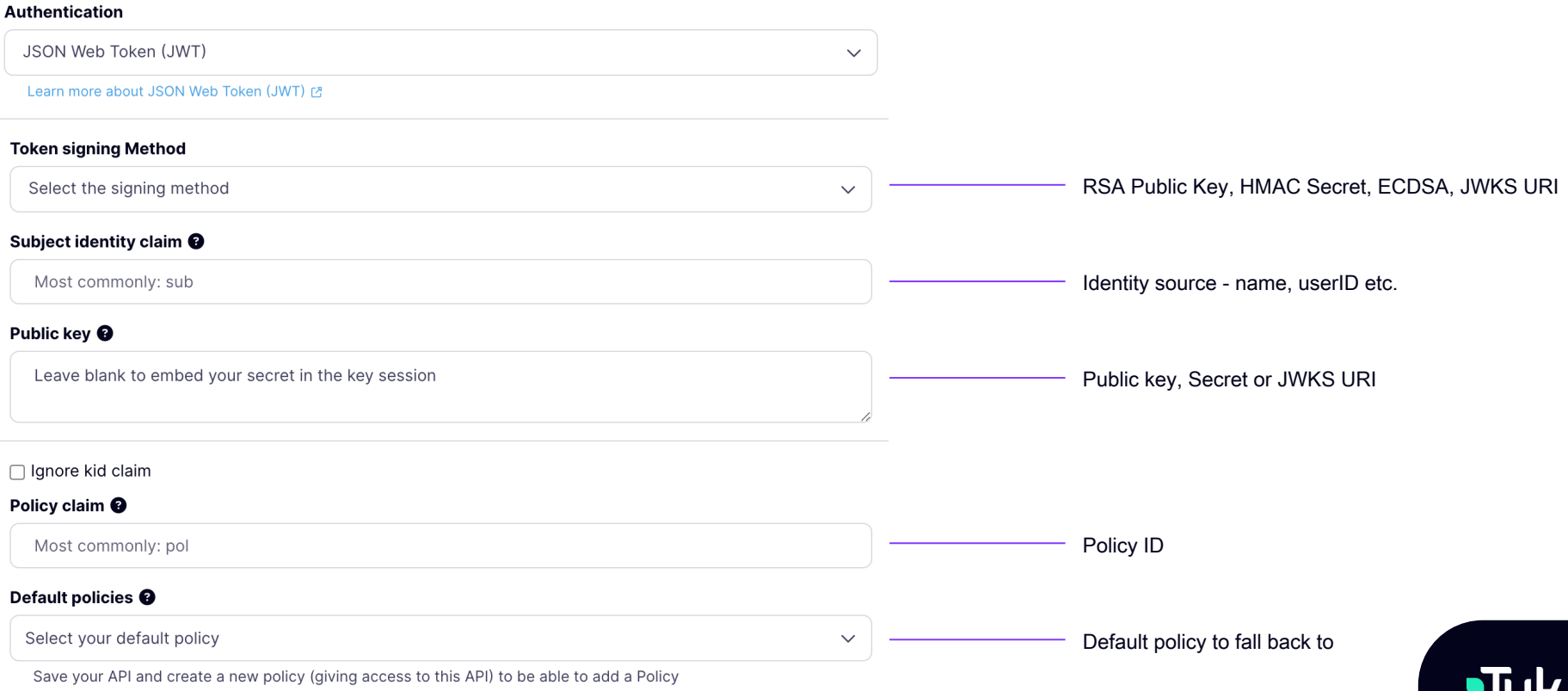
- Cryptographically signed
- Claims signed by 3rd party
- Configure Tyk to extract user id and policy id

```
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsIn.eyJzdWIiOiIxMjNTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiYWRt.TJVA95OrM7E2cBab30R
MHrH
```

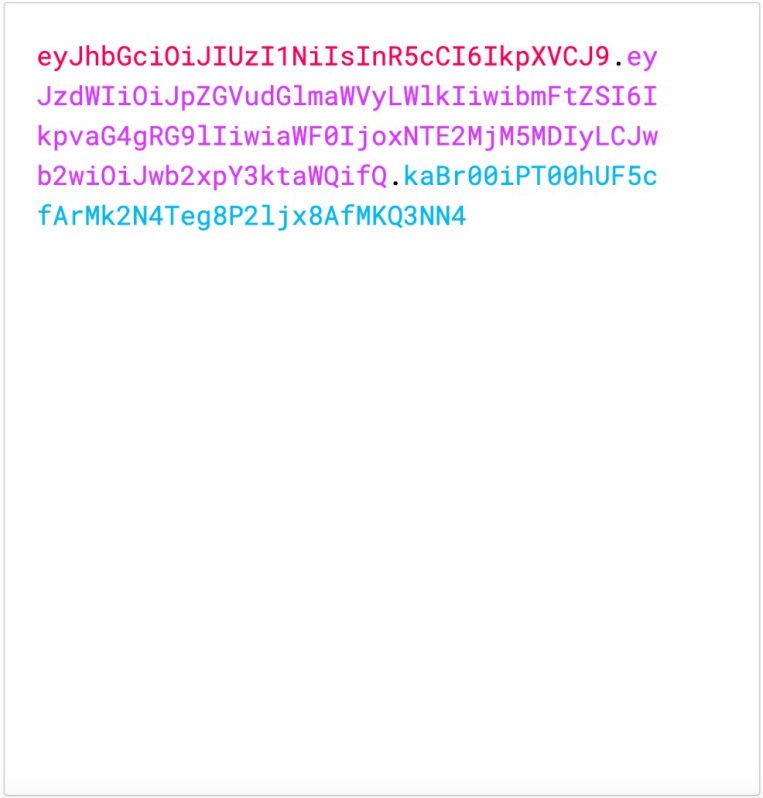
Tyk


3. Generates JWT with custom policy and identity claims
Alice
1. Logs in via
App / website
2. Uses a custom JWT provider to generate claims
3rd party / custom IDP
4. Uses JWT with gateway
Tyk gateway
5. Tyk generates an internal identity representation based on identity claim
6. Tyk applies the access policy stated in the policy claim to Alice's internal identity representation
7. Tyk proxies request to target

JSON Web Tokens



JSON Web Tokens



OAuth 2.0

- OAuth 2.0 is an open, industry-standard protocol for authorization.
- It enables secure delegated access to APIs without sharing user credentials.
- Based on RFC 6749, it supports third-party apps acting:
  - On behalf of a user (Resource Owner)
  - On their own behalf (e.g., backend services)
- Instead of passwords, OAuth relies on Access Tokens for API access.
- Common use cases:
  - Apps integrating with Google, Microsoft, Facebook APIs
  - Allowing third-party access to enterprise APIs securely

Tyk

OAuth 2.0

Client → requests permission from → **User (Resource Owner)** → logs into → **Authorization Server** → issues → **Access Token** → used to access → **Resource Server**

(Needs diagram)

Tyk

Term	Description
Protected Resource	The API or service that needs authorization (e.g., a user's data)
Resource Owner	The user or system owning the Protected Resource
Client	App or system requesting access to the resource
Access Token	Temporary key proving access permission, passed in API calls
Authorization Server	Issues tokens after authenticating and authorizing the Client
Client Application	The app registered with the Authorization Server to request access
Resource Server	The API backend that hosts the data; it checks the token
Identity Server	Manages user authentication (can be the same as Authorization Server)
Scope	Defines what the Client can access (e.g., "read", "write")
Grant Type	The OAuth flow used (e.g., Authorization Code, Client Credentials)

Tyk

OAuth 2.0

Understanding Access Tokens & Client Registration

Access Tokens

- Represent the authorization granted by the Resource Owner to the Client
- Typically opaque strings or JWTs (which may include user identity, scopes, and expiry)
- Sent with each API request to authenticate the client
- Usually have an expiry time to limit access duration
- Can be refreshed using a Refresh Token (if supported)

Client Application Registration

To request an Access Token, a client must register with the Authorization Server:

- Client ID – Public identifier for the app
- Client Secret – Confidential key shared with the server
- Redirect URI – Where the user is sent after login
- Client authenticates using ID and Secret during token request
- Authorization Server validates and issues an Access Token

Tyk

OAuth 2.0

Tyk can act as an OAuth 2.0 Authorization Server

Tyk handles:

- Token generation & management
- Access control for client apps

Key Features:

- Fine-Grained Access Control
  - Use policies, versioning, and API IDs to restrict access.
- Usage Analytics
  - Track and analyze usage by Client ID.
- Multi-API Access
  - One OAuth token can access multiple APIs
  - (e.g., issue token via API A, access API B via Auth Token).

OAuth 2.0

Tyk supports these OAuth 2.0 grant types:

1. **Authorization Code Grant**
    a. User is redirected to identity server
    b. Requires user approval before access token is issued

1. **Client Credentials Grant**
    a. For machine-to-machine authentication
    b. Uses client ID + client secret

1. **Resource Owner Password Grant (legacy use only)**
    a. Client uses user's own credentials to authenticate
    b. ⚠️ Not recommended — considered insecure
    c. Provided for legacy compatibility only

OAuth 2.0

Manage Client Access Policies

- Access tokens issued by Tyk are **standard session objects**
- Can be bound to security policies at the time of token creation
- Policies can define:
  - Rate limits
  - Quotas
  - Access rights

- Policies applied to a **Client App** affect all tokens issued for it

Client App Registration

- All grant types start with registering a Client App in the Tyk Dashboard
- Registration generates:
  - Client ID
  - Client Secret
- These credentials are required in future token requests